

3일 차

회귀 모델링

이번 차시에서 주목할 키워드

- 회귀 모델
- 손실 함수(MSE)
- 옵티마이저
- 학습률
- 학습 곡선

3일 차 학습 목표

- 회귀 모델의 개념과 구조를 이해하고 직접 설계하며 모델링의 흐름을 익힙니다.
- 손실 함수, 옵티마이저, 학습률, 배치 크기 등 주요 하이퍼파라미터를 이해합니다.
- 학습 곡선과 검증 곡선을 해석하며 모델이 제대로 학습하고 있는지를 배웁니다.

회귀 모델링(Regression Modeling)이란 숫자를 예측하는 모델을 만드는 과정입니다. 이번 차시에서는 딥러닝으로 회귀 모델링을 수행하기 위한 기본적인 사항들을 살펴봅니다.

2일 차에서 익힌 딥러닝의 기본 구조와 학습 흐름을 바탕으로 실제 데이터를 이용해 회귀 모델을 처음부터 끝까지 구성합니다. 모델 설계부터 학습, 평가에 이르기까지 실습 중심으로 진행하므로 예제를 따라하면 딥러닝 회귀 모델이 어떤 방식으로 숫자 데이터를 예측하는지 자연스럽게 이해할 수 있습니다. 특히 이번 차시에서 다루는 손실 함수, 옵티마이저, 학습률, 배치 크기, 에포크 등의 개념은 이후 더 복잡한 모델을 다룰 때도 반복해서 사용하므로 반드시 익히기를 바랍니다.

딥러닝 모델 구조 설계하기

딥러닝 모델 구조를 어떻게 설계하는지 모델 구조를 구성하는 요소는 무엇인지 설명합니다.

프로세스의 이해

딥러닝 모델 구조를 이해하기 전에 먼저 ‘프로세스(Process)’라는 개념을 간단히 알아보겠습니다. 프로세스란 어떤 일을 단계적으로 수행하는 흐름을 말합니다. 이 단계를 이해하면 모델이 어떻게 작동하는지 더 쉽게 받아들일 수 있습니다.

프로세스란 일정한 목표를 이루기 위해 하나의 작업(Task)이 다른 작업(Task)과 연결되면서 서로 협력하는 일련의 단계들을 말합니다. 각각의 단계는 이전 단계에서 얻은 출력(Output)을 받아 새로운 입력(Input)으로 사용하며 이를 처리한 결과를 다음 단계로 전달합니다. 따라서 프로세스에서 각각의 단계는 독립적으로 동작하는 것이 아니라 공통의 목표를 이루기 위해 긴밀히 연결되어 있습니다. [그림 3-1]은 일반적인 프로세스를 다이어그램으로 보여 주고 있습니다.



[그림 3-1] 일반적인 프로세스

여기서 각각의 단계는 자신만의 고유한 Task를 수행하고, 처리된 결과는 다음 단계의 입력으로 활용됩니다. [그림 3-1]과 같이 Task1의 Output은 Task2의 Input으로 전달됩니다. 기업의 상품 생산 프로세스를 예로 들겠습니다. 상품의 기획 단계에서 아이디어가 만들어지면 디자인 단계에서 상품의 외형 설계 및 자재가 결정되고 생산 단계에서 실제 제품이 만들어집니다. 그 후 물류 입고, 매장 판매 과정을 거쳐 최종적으로 소비자에게 도달합니다.

딥러닝도 이와 같은 프로세스의 개념과 유사합니다. 모델은 다음과 같은 순서로 학습합니다.

- ① Input 데이터를 받아들입니다(예: 테이블 형태 데이터, 이미지, 텍스트 등).
- ② 데이터를 여러 층(Layer)에서 처리합니다(예: Linear, CNN, RNN Layer 등).
- ③ 최종적으로 Output을 생성합니다(예: 분류 결과, 회귀값 등).

이 학습 과정은 단 한 번으로 끝나는 것이 아닙니다. 모델은 반복적인 학습으로 점차 개선됩니다. 마치 공부할 때 교재를 여러 번 반복 학습하는 것처럼 모델 또한 주어진 데이터를 반복 학습하며 점점 더 정확한 결과를 내는 방향으로 성능을 개선합니다.

프로세스가 단순히 단계의 나열이 아니라 단계가 서로 연결되어 하나의 시스템으로 완성되듯이, 딥러닝도 입력에서 시작해 여러 층을 거쳐 출력층에 이르는 전체적인 구조가 유기적으로 이어지면서 비로소 하나의 모델이 됩니다.

이제 프로세스의 기본 개념을 바탕으로 회귀 모델링에서는 모델 구조를 어떻게 설계하고, 손실 함수와 옵티마이저는 어떻게 선택하는지 구체적으로 알아보겠습니다.

딥러닝 모델의 기본 요소들

딥러닝 모델은 입력(Input) 데이터를 받아 여러 연산 단계(Layer)를 거쳐 출력(Output)을 생성하는 구조입니다. 이 구조를 설명하기 위해서 ‘보스턴(Boston) 집값’을 예측하는 간단한 회귀 모델을 예로 들겠습니다.

데이터 소개

보스턴 집값 데이터는 1970년대 미국 보스턴 시의 주택 가격을 설명하기 위해 수집된 대표적인 회귀 분석용 데이터입니다. 범죄율, 주택당 평균 방의 개수, 고속도로 접근성, 교육 수준 등과 같은 다양한 특징이 집값 데이터에 포함되어 있는데, 이 특징들을 이용해 주택의 중위 가격을 예측하는 데 활용합니다.

‘보스턴 집값’ 데이터의 변수는 다음과 같습니다.

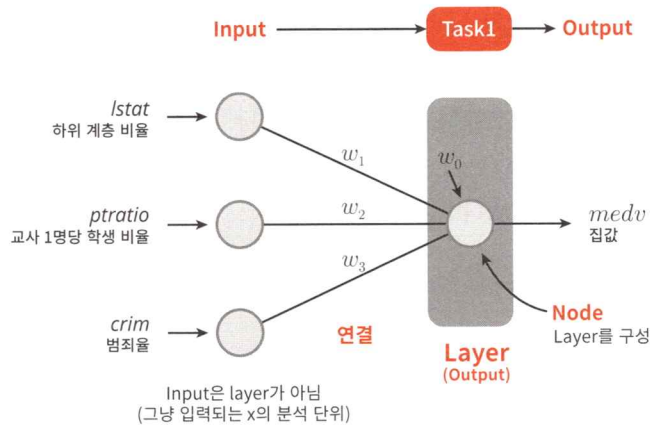
- crim: 지역별 범죄율(특정 지역의 1인당 범죄 발생 비율)
- zn: 25,000평방피트 이상의 주거 구역 비율
- indus: 비소매 상업 지역 면적 비율(상업용 부동산이 차지하는 비율로서 상업 지역의 밀도)
- chas: 찰스강 인접 여부(1은 인접, 0은 비인접)
- nox: 일산화질소 농도(대기 중 일산화질소의 농도를 나타내는 대기 오염 지표)
- rm: 주택당 평균 방의 개수
- age: 1940년 이전에 건축된 주택 비율
- dis: 5개 보스턴 고용센터까지의 거리(중심 업무 지구까지의 가중 평균 거리)
- rad: 주요 고속도로 접근성 지수(값이 클수록 접근성이 높음)
- tax: 재산세율(단위는 10,000달러당 세금)
- ptratio: 학생-교사 비율(한 교사가 담당하는 평균 학생 수로서 교육 여건 평가 지표)
- lstat: 하위 계층 비율
- medv: 타운별 주택의 중앙값으로 단위는 1,000달러(target)

이 데이터를 이용해 간단한 딥러닝 모델부터 점차 복잡한 구조의 모델까지 다뤄보겠습니다.

간단한 딥러닝 모델 구조

보스턴 집값 데이터의 모든 특징(feature)을 이용해 모델링할 수 있지만, 좀 더 쉽게 설명하기 위해 단순한 구조에서 시작합니다.

[그림 3-2]의 딥러닝 모델은 입력(Input)과 층(Layer) 그리고 출력(Output)으로 구성되어 있습니다. 모델 구조에서 동그라미는 노드(Node)로서 하나의 정보를 의미합니다. 각각의 요소를 하나씩 살펴보겠습니다.



[그림 3-2] 간단한 회귀 모델 구조

모델 입력

모델 입력(Input)으로 다양한 특징(feature)을 사용할 수 있습니다. [그림 3-2]에서는 세 가지 정보인 *lstat*, *ptratio*, 그리고 *crim*을 입력으로 지정해서 집값(*medv*)을 예측하려고 합니다. 입력 데이터는 모델의 첫 번째 층으로 전달(또는 연결이라고도 함)됩니다.

층

모델에 입력된 데이터는 첫 번째 층(Layer)과 연결되며, 이때 각각의 입력값에 가중치(Weight)를 곱해 입력의 중요도를 조절합니다. [그림 3-2]의 구조에서 층은 1개이고 노드(Node)도 1개입니다. 층의 노드 수는 층의 출력값 수입니다. 여기서 층의 입력 정보는 3개이고 출력 정보는 1개입니다.



여기서 잠깐

노드를 부르는 다양한 이름들

딥러닝에서 사용하는 노드(Node), 뉴런(Neuron), 유닛(Unit)은 거의 같은 개념을 가리키지만, 맥락에 따라 조금씩 다르게 표현됩니다.

- 뉴런(Neuron): 생물학적 신경세포에서 따온 이름으로 입력을 받아 가중치와 합산한 다음, 활성화 함수를 거쳐 출력하는 연산 단위를 뜻합니다.
- 노드(Node): 그래프 구조 관점에서 본 표현으로 신경망을 수학적으로 모델링했을 때 각각의 연산 지점을 가리키는 데 주로 사용합니다.
- 유닛(Unit): 코드에서는 뉴런 혹은 노드를 유닛으로 표현하기도 합니다.

본질적으로 같은 개념을 지칭하지만 표현 방식이 약간씩 다르다고 이해하면 됩니다.

가중치는 어떤 연결 정보가 더 중요한지 나타내는 값입니다. 모델 학습 과정에서는 이 가중치들을 적절히 조정해 보다 정확한 결과를 생성합니다. 층의 노드에서는 이런 입력값과 가중치를 이용해서 연산을 수행합니다. 즉, 입력 데이터(*lstat*, *ptratio*, *crim*)는 각기 다른 가중치(w_1 , w_2 , w_3)와 곱셈 연산을 하고 합산됩니다. 다음은 노드의 연산을 표현하는 수식입니다.

$$medv = w_1 \cdot lstat + w_2 \cdot ptratio + w_3 \cdot crim + w_0$$

수식 말미에 있는 w_0 은 바이어스(Bias)로서 모델이 데이터를 보다 유연하게 학습하도록 돕는 값입니다. 바이어스는 그래프에서 직선의 시작 위치를 옮겨 주는 역할을 합니다. 만약 바이어스가 없다면 모델이 학습하는 선이나 곡선은 무조건 원점을 지나가야 합니다. 그러면 데이터가 원점 근처에 있지 않을 때는 아무리 가중치를 잘 조정해도 제대로 예측하기 어렵습니다.

입력값과 가중치의 곱셈 결과는 층의 각 노드에서 처리됩니다. 층은 딥러닝 모델 구조의 기본 단위로서 데이터를 처리하고 그 결과를 다음 단계로 전달하는 역할을 합니다. 층은 여러 노드로 구성할 수 있는데, 각각의 노드는 다음과 같은 절차로 자신의 역할을 수행합니다.

- ① 입력 데이터와 가중치를 받아들입니다.
- ② 주어진 수학적 계산(예: 가중치와 입력값의 곱셈과 합산)을 수행합니다.
- ③ 결과를 다음 층이나 최종 출력으로 전달합니다.

출력

모델의 최종 결과는 출력(Output)으로 나타납니다. [그림 3-2]에서는 출력값이 집값(*medv*)에 대한 예측값이므로 모델은 입력 데이터(*lstat*, *ptratio*, *crim*)를 이용해 가장 적합한 집값을 예측하기 위해 학습합니다. 즉, 모델 학습 과정은 각각의 가중치를 최적화하며 예측의 정확도를 높이는 데 초점을 맞춥니다.

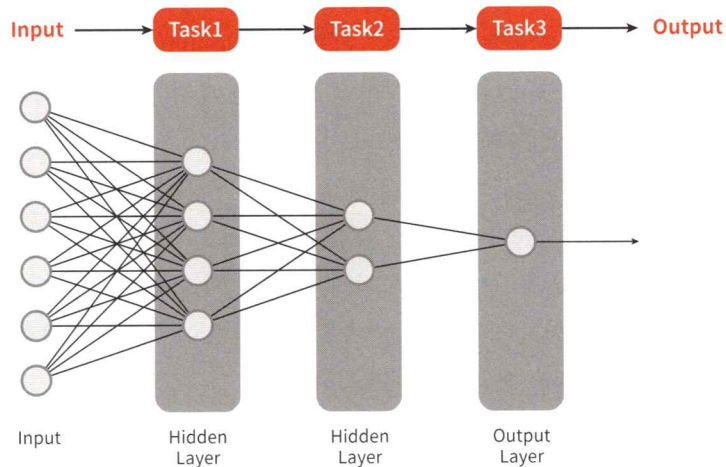
다층 구조

[그림 3-2]는 단층 구조지만, 딥러닝은 보통 층(Layer)을 여러 개로 구성합니다. 여러 층으로 구성하면 모델은 정보를 점차 추상화해 복잡한 패턴을 학습할 수 있습니다. 이와 같은 구조를 다층 신경망(Multi-Layer Neural Network) 또는 간단히 다층

(Multi-Layer) 구조라고 합니다. 전통적인 머신러닝에서는 하나의 입력층과 출력층으로 이루어진 단순한 모델이 많았지만, 딥러닝에서는 은닉층(Hidden Layer)을 여러 개 추가함으로써 훨씬 더 정교한 표현과 학습이 가능합니다.

왜 이렇게 여러 층이 필요한 걸까요? 그 이유는 바로 현실 세계의 데이터가 생각보다 복잡하고 비선형적인 구조이기 때문입니다. 이미지, 음성, 텍스트와 같은 데이터는 단순한 선형 모델로는 잘 설명되지 않습니다. 따라서 딥러닝은 층을 깊게 쌓아 단순한 특징에서 점점 더 복잡하고 추상적인 특징으로 발전시키는 구조입니다. 이 과정에서 머신(컴퓨터)은 사람처럼 복잡한 패턴을 인식하고 예측하는 능력을 키울 수 있습니다.

[그림 3-3]은 다층 구조를 다이어그램으로 나타냅니다.



[그림 3-3] 다층 구조(Multi-Layer) 예시

[그림 3-3]의 윗부분은 일반적인 데이터 처리 프로세스입니다. 입력(Input)이 주어지면 순차적으로 Task1, Task2, Task3이라는 단계를 거쳐 최종 결과(Output)를 만듭니다. 이때 각각의 작업(Task)은 이전 작업의 결과를 바탕으로 수행됩니다. 단계적인 작업 흐름은 딥러닝 모델에서도 그대로 적용됩니다.

[그림 3-3]의 아랫부분은 실제 딥러닝 신경망 구조를 단순화한 도식입니다. 입력(Input)으로 시작된 데이터는 두 개의 은닉층(Hidden Layer)을 차례로 거쳐 출력층(Output Layer)에 도달합니다. 각각의 은닉층에서는 노드들이 이전 층의 출력을 받아 가중치를 곱하고 비선형 함수를 적용해 새로운 출력을 만듭니다. 이 과정에서 모델은 점점 더 복잡한 규칙과 특징을 학습합니다.

딥러닝의 다층 구조는 일종의 계층적 학습 구조입니다. 각각의 층은 자신에게 주어진 입력을 바탕으로 단순한 규칙을 학습하고 그 결과를 다음 층으로 전달합니다. 각각의 층이 작은 단위의 학습을 반복함으로써 전체 모델은 간단한 패턴 → 중간 수준의 구조 → 고차원의 추상적인 특징을 발전시킵니다.

예를 들어 이미지 데이터를 다룬다고 가정하면 딥러닝 모델은 첫 번째 은닉층에서는 선이나 점과 같은 기본적인 패턴을 인식하고, 두 번째 은닉층에서는 눈, 코, 입 같은 중간 단위의 구성 요소를 파악하며, 마지막 층에서는 얼굴 전체를 종합적으로 판단하는 식으로 동작합니다.

은닉층을 추가하면 모델의 표현 능력(Representation Power)이 증가해 복잡한 문제를 더 잘 해결할 수 있습니다. 하지만 여기에는 주의할 점도 있습니다. 층이 많아지면 학습 시간이 오래 걸릴 뿐만 아니라 너무 복잡한 구조로 인해 과적합(Overfitting) 문제를 일으킬 수 있습니다. 또한 기울기 소실(Vanishing Gradient)이라는 학습 문제도 발생할 수 있습니다. 물론 이런 문제들을 해결하기 위한 여러 방법들이 개발되었습니다. 적절한 활성화 함수의 사용(예: ReLU), 드롭아웃(Dropout) 같은 규제 기법 활용, 배치 정규화(Batch Normalization) 등을 통해 모델의 안정성과 일반화 성능을 높일 수 있습니다.

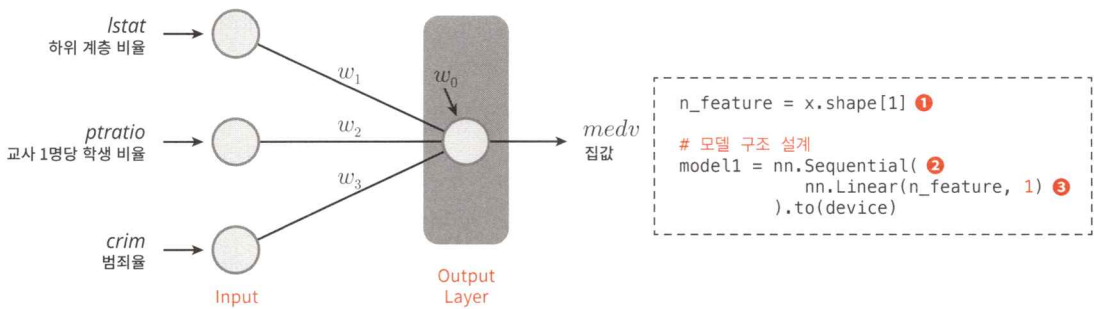
이들 방법의 세부 기법과 구현은 이후 차시에서 하나씩 다룹니다. 여기서는 딥러닝은 여러 층으로 이루어져 정보를 단계적으로 처리하며 이 과정에서 점점 더 깊고 정교한 패턴을 학습한다는 점만 기억하면 됩니다.

모델 구조와 코드 비교

딥러닝 모델의 구조를 파이토치 코드로 살펴보겠습니다. 딥러닝 모델은 입력 데이터에서 출력 결과를 생성하기 위해 설계되며, 이를 수학적으로 정의하고 코드로 표현하는 과정이 필수적입니다.

[그림 3-4]에서는 모델과 코드를 비교하면서 서로 어떻게 연관되는지 살펴보겠습니다.

- ❶ `n_feature = x.shape[1]`: 입력 데이터 `x`의 열(Column) 수를 가져옵니다. [그림 3-4]의 모델은 입력 정보가 3개(`lstat`, `ptratio`, `crim`)입니다.
- ❷ `nn.Sequential()`: 딥러닝 모델 구조를 설계할 때 여러 층을 순서대로 연결해 하나의 모델로 정의하는 함수입니다.



[그림 3-4] 간단한 모델 구조와 코드

③ `nn.Linear(n_feature, 1)`: 하나의 층(Layer)을 정의하는 함수입니다. 첫 번째 인수는 입력 특징 수(3개), 두 번째 인수는 출력 노드 수(1개)를 나타냅니다. 이 층은 입력과 가중치의 곱셈과 바이어스의 합산을 수행한 다음, 출력을 생성합니다.

손실 함수

앞서 정의한 모델 선언(모델 구조 설계) 코드에 이어 손실 함수(Loss Function)를 지정합니다. 모델을 설계하는 시점에서 어떤 손실 함수를 사용할지 지정하며, 모델 학습 단계에서 이 손실 함수를 사용합니다.

CODE

```
loss_fn = nn.MSELoss() # 손실 함수 정의
```

손실 함수는 다른 이름으로 오차 함수, 비용 함수, 목적 함수라고도 합니다. 딥러닝 모델 학습은 손실 함수로 모델이 얼마나 잘 작동하는지 평가하고 이를 개선하는 작업이 핵심입니다. 개선 과정은 모델이 예측한 값과 목표값의 차이(오차)를 줄이기 위해 가중치를 조금씩 조정하며 진행됩니다. 손실 함수는 회귀 모델인 경우 MSE(Mean Squared Error), 분류 모델인 경우 교차 엔트로피(Cross Entropy)를 사용합니다. 이번 코드에서는 회귀 모델의 손실 함수인 MSE(`nn.MSELoss`)를 사용했습니다.

MSE는 목표값(y)과 예측값($\hat{y}$)의 차이를 제곱해 계산합니다. 그런데 MSE 수식을 보면 제곱 오차에 1/2을 곱하고 있습니다.

$$\text{nn.MSELoss}() = \frac{1}{2} \|y - \hat{y}\|^2$$

수식은 모델 최적화 과정에서 손실 함수를 미분하게 되는데, 이때 제곱의 2가 내려와 1/2과 곱하면서 1로 상쇄되도록 해줍니다.

일반적으로 오차(error)의 크기를 계산하는 방법은 두 가지입니다. 즉, 제곱 오차(Squared Error)나 절댓값 오차(Absolute Error)를 이용해 구합니다. 제곱 오차를 L2 오차(손실), 절댓값 오차를 L1 오차(손실)라고 합니다. 분류 모델의 손실 함수인 교차 엔트로피의 공식과 설명은 다음 차시인 분류 모델링에서 다룹니다.

옵티마이저

딥러닝 모델 학습은 손실 함수로 모델의 성능을 평가하고 옵티마이저(Optimizer, 최적화기)를 활용해 손실값을 줄이는 방향으로 모델의 가중치와 바이어스를 업데이트하는 과정입니다. 옵티마이저는 학습 과정에서 손실값을 최소화할 수 있도록 안내하는 역할을 합니다. 옵티마이저는 손실 함수에서 계산된 오차를 기반으로 그라디언트(Gradient, 기울기)를 계산해 가중치를 조정합니다.

옵티마이저도 다음과 같은 코드로 선언합니다.

CODE

```
optimizer = Adam(model1.parameters(), lr=0.1)
```

옵티마이저가 모델의 모든 가중치와 바이어스(model1.parameters())를 업데이트하도록 지정합니다. 이때 lr=0.1과 같이 가중치를 얼마나 조절할지에 영향을 주는 비율(학습률)을 함께 설정합니다.

코드에서 사용한 Adam(Adaptive Moment Estimation)은 딥러닝에서 가장 널리 사용되는 옵티마이저입니다. Adam은 학습 과정에서 학습률을 동적으로 조정하고 이전 기울기 정보를 활용해 업데이트 방향의 안정성을 높입니다. 2일 차에서 이를 모멘텀(Momentum) 기법이라고 했습니다. 또한 옵티마이저는 기울기 변화가 크더라도 학습이 안정적으로 진행되도록 합니다.



옵티마이저의 발전

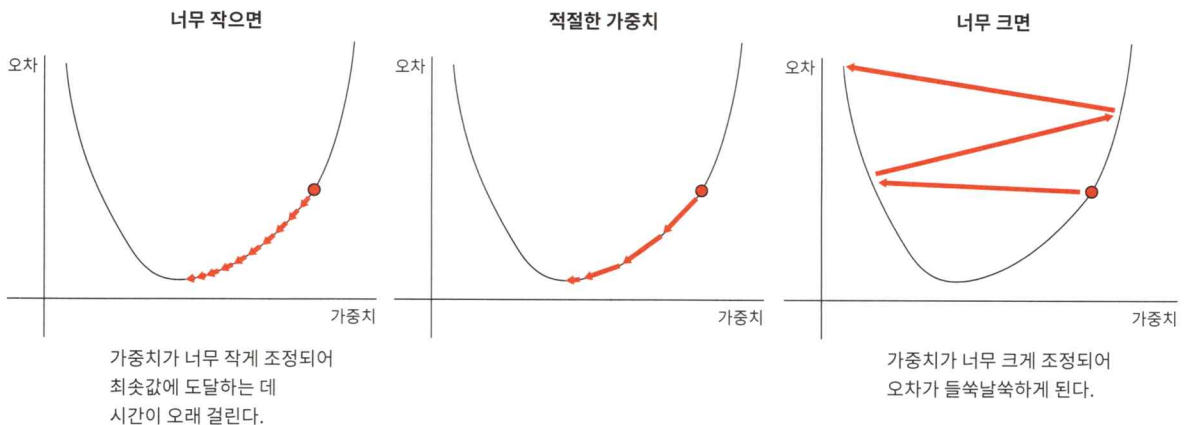
초기에는 가장 단순한 방법인 경사 하강법(Gradient Descent)이 널리 사용되었습니다. 이 방법은 손실을 줄이기 위해 기울기의 반대 방향으로 조금씩 이동하며 가중치를 업데이트합니다. 하지만 이 방식은 학습 속도가 느리고 복잡한 손실 함수에서는 쉽게 최적화되지 않는 문제가 있습니다. 경사 하강법을 보완해 좀 더 빠르게 학습시키고 학습 안정성을 확보하려는 방법들이 여럿 고안되었는데, 다음과 같은 옵티마이저들이 대표적입니다.

- SGD(Stochastic Gradient Descent): 확률적 경사 하강법. 반복할 때마다 전체 데이터가 아닌 일부 데이터(Batch)를 사용해 가중치를 조정하므로 계산이 빠릅니다.
- Momentum: SGD에 속도를 더해 주는 방식. 이전의 이동 방향을 일정 비율로 유지하면서 더 빠르고 부드럽게 학습하도록 도와줍니다.
- Adagrad: 파라미터마다 학습률을 다르게 적용하는 방식. 자주 업데이트되는 파라미터는 작은 학습률, 드물게 업데이트되는 파라미터는 큰 학습률을 사용합니다. 하지만 시간이 지날수록 학습률이 너무 작아지는 문제가 있습니다.
- RMSprop: Adagrad의 단점을 보완해 최근의 기울기를 중심으로 학습률을 조정합니다. 특히 RNN 계열 모델에서 성능이 좋습니다.
- Adam(Adaptive Moment Estimation): Momentum과 RMSprop을 결합한 방식으로 현재 가장 널리 사용되는 옵티마이저입니다. 학습률을 자동으로 조절하면서도 빠르고 안정적인 수렴을 제공합니다.

학습률

학습률(Learning Rate)은 옵티마이저가 가중치를 얼마나 빠르게 업데이트할지 결정하는 중요한 하이퍼파라미터(Hyperparameter)입니다. [그림 3-5]는 학습률에 따라 오차가 어떻게 줄어드는지 보여 줍니다. 각각의 그래프에서 y축은 손실, x축은 가중치입니다.

[그림 3-5]에서 화살표는 가중치를 조정했을 때 오차가 줄어드는 방향과 크기입니다. 맨 오른쪽 그래프처럼 학습률이 너무 크면 학습 과정이 불안정해지고 최적의 손실값에 도달하지 못할 수 있습니다. 맨 왼쪽 그래프처럼 학습률이 너무 작으면



[그림 3-5] 학습률에 따라 오차가 줄어드는 정도 비교

학습 속도가 매우 느려져 시간이 많이 걸릴 수 있습니다. Adam 옵티마이저는 기본적으로 가중치가 조정되어 감에 따라 학습률도 조절되는 메커니즘을 포함하고 있지만, 초기 학습률은 사용자가 설정해야 합니다. 보통 처음에는 0.001이나 0.01을 많이 사용합니다.

회귀 모델링 실습 1: 단일 층으로 이루어진 회귀 모델

이제 새 코랩 파일을 만들고 앞서 설계한 구조로 코드를 작성하겠습니다. 새로 작성한 코랩 파일의 이름은 '3일차_회귀모델실습1.ipynb'로 명명합니다.

환경 준비에서 데이터 전처리까지

이미 여러 번 반복 설명한 코드들은 따로 설명하지 않습니다. 기억나지 않는다면 앞에서 설명한 내용을 다시 한번 참고해 코드를 작성하길 바랍니다.

환경 준비: 라이브러리 불러오기

주요 라이브러리를 먼저 불러옵니다.

```
CODE
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import *
from sklearn.preprocessing import StandardScaler, MinMaxScaler
```

다음 코드 셀에서 파이토치 라이브러리와 관련 함수들도 불러옵니다.

```
CODE
import torch ①
from torch import nn ②
from torch.utils.data import DataLoader, TensorDataset ③
from torch.optim import Adam ④
```

- ① 파이토치의 기본 함수 전체를 불러옵니다.
- ② 신경망(nn, neural network) 관련 모듈이 포함된 함수들을 불러옵니다.
- ③ 텐서 데이터 세트와 데이터 로더를 선언하기 위한 함수를 불러옵니다.
- ④ Adam 옵티마이저 함수를 불러옵니다.

파이토치 함수를 불러오는 코드는 자주 사용하므로 눈에 익숙해져야 합니다.

환경 준비: 디바이스 설정

코랩에서는 생성하는 파일마다 GPU 혹은 CPU를 설정할 수 있는데, 기본 설정은 CPU입니다. 코랩 파일에 설정된 디바이스를 확인하고 준비하는 코드를 다음과 같이 작성합니다.

CODE

```
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using {device} device")
```

OUTPUT

```
Using cpu device
```

코드를 실행하면 Using cpu device가 출력되는 것을 확인할 수 있습니다(이 책에서 GPU는 꼭 필요할 때에 한정해 사용합니다).

데이터 로딩 및 준비

데이터를 불러와 전처리합니다. 데이터 전처리는 x, y를 분할하고, 학습 데이터(train)와 검증 데이터(val)를 분할하고 스케일링하는 작업입니다. 이번 모델의 특징(feature)에는 범주형 데이터가 없으므로 가변수화는 하지 않습니다.

CODE

```
# 데이터 로딩
path = 'https://bit.ly/ds_boston_csv' ①
data = pd.read_csv(path)

# x, y 분할
target = 'medv'
features = ['lstat', 'ptratio', 'crim'] ②
x = data.loc[:, features] ③
y = data.loc[:, target]

# train, val 분할
x_train, x_val, y_train, y_val = train_test_split(x, y, test_size=.2, random_state = 20)

# 스케일러 선언
scaler = MinMaxScaler()
x_train = scaler.fit_transform(x_train)
x_val = scaler.transform(x_val)
```

| ① 보스턴 집값 데이터(csv 파일)의 주소 링크입니다.

- ② 모델 구조에 맞게 전체가 아니라 lstat, ptratio, crim 세 개의 입력변수(feature)만 사용합니다.
- ③ 세 개의 입력변수만 담는 x의 데이터 세트를 구성합니다.

데이터 로더와 사용자 정의 함수

이제 데이터 세트에 대한 기본 준비가 끝났으므로 학습할 때 미니배치 단위로 데이터를 공급하는 데이터 로더를 준비하겠습니다.

2일 차에서 설명했듯이 딥러닝 모델 학습에서는 전체 데이터를 한꺼번에 입력하기보다는 미니배치(Mini Batch, 작은 일괄처리 단위) 단위로 나누어 입력하고 학습합니다.

이때 배치 크기는 모델의 학습 성능과 효율성에 영향을 미칩니다. 배치 크기가 크면 한 번의 학습에서 많은 데이터를 활용하므로 모델의 가중치 업데이트가 더 안정적입니다. 그러나 메모리 사용량이 증가하고 학습 시간이 길어질 수 있습니다. 배치 크기가 작으면 업데이트가 더 자주 이루어져 학습이 빠르게 진행되지만, 가중치 조정에서 변동성이 커질 수 있습니다. 배치 크기의 기본값으로 32를 많이 사용합니다.

딥러닝 학습에서 데이터 로더를 만들려면 다음과 같은 절차로 진행해야 합니다.

- x, y를 텐서로 바꾼다.
- x 텐서, y 텐서를 텐서 데이터 세트로 만든다.
- 만든 텐서 데이터 세트로 데이터 로더를 선언한다.

모델링 과정에서 자주 반복되는 코드는 사용자 정의 함수로 만들어 두었다가 필요할 때 사용하면 작업이 훨씬 효율적입니다. 여기서는 데이터 로더의 생성을 한꺼번에 처리해 주는 make_Dataset이라는 사용자 정의 함수를 하나 만들겠습니다.

CODE

```
def make_DataSet(x_train, x_val, y_train, y_val, batch_size = 32) : ①

    # 데이터 텐서로 변환
    x_train_tensor = torch.tensor(x_train, dtype=torch.float32) ②
    y_train_tensor = torch.tensor(y_train, dtype=torch.float32).view(-1, 1) ③
    x_val_tensor = torch.tensor(x_val, dtype=torch.float32)
    y_val_tensor = torch.tensor(y_val, dtype=torch.float32).view(-1, 1)
```

```
# TensorDataset 생성: 텐서 데이터 세트로 합치기
train_dataset = TensorDataset(x_train_tensor, y_train_tensor) ④

# DataLoader 생성
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle = True) ⑤
return train_loader, x_val_tensor, y_val_tensor ⑥
```

- ① make_DataSet 함수는 x_train, x_val, y_train, y_val 데이터 세트와 batch_size(미니 배치의 크기)를 입력으로 받습니다. 이때 x_train, x_val, y_train, y_val은 모두 넘파이 배열 형식이어야 텐서로 변환할 수 있습니다. batch_size의 기본값은 32로 지정합니다.
- ② torch.tensor 메서드로 x_train을 텐서로 변환합니다. x_train을 실수형 데이터로 처리 하도록 타입은 dtype=torch.float32로 지정합니다. 보통 입력값(x)은 실수형으로 지정합니다.
- ③ y_train의 1열에 데이터를 맞추도록 구조를 변환(view(-1, 1))합니다. 결과값을 2차원 구조로 만든다는 의미입니다.
- ④ TensorDataset 함수는 두 개 이상의 텐서를 하나의 묶음으로 다룰 수 있게 해줍니다. 즉, x_train_tensor와 y_train_tensor가 한쌍으로 연결된 데이터가 되어 학습에서 사용됩니다.
- ⑤ DataLoader 함수는 학습할 때 사용할 수 있게 데이터를 미니배치 단위로 나눕니다. batch_size를 지정하고 shuffle=True로 데이터를 섞은 다음, 배치를 만들어 공급합니다.
- ⑥ 학습 데이터 로더인 train_loader와 검증 데이터 세트(텐서)인 x_val_tensor, y_val_tensor를 결과로 반환합니다.

딥러닝 모델을 제대로 학습시키려면 입력값(feature, x)과 목표값(target, y)의 데이터 타입과 차원을 정확히 확인하는 일이 매우 중요합니다. 특히 x와 y의 형태는 문제 유형에 따라 다르게 구성되는데, 이를 잘못 설정하면 오류가 발생합니다.

먼저 입력값 x는 대부분 실수형(float)으로 처리하며 2차원 구조입니다. 목표값 y는 문제의 종류에 따라 데이터 타입과 차원이 달라집니다. 회귀 문제에서는 숫자를 예측하므로 목표값도 실수형이며 2차원 형태로 구성합니다. 반면 분류 문제는 목표값이 클래스 번호(예: 0, 1, 2 등)로 주어지는데, 이때는 정수형으로 처리하고 1차원 형태로 구성해야 합니다.

요약하면 x는 항상 실수형의 2차원 배열이고 y는 회귀 문제의 경우 실수형 2차원 텐서, 분류 문제의 경우 정수형 1차원 텐서로 구성한다는 점을 꼭 기억해야 합니다. 자료형과 차원을 올바르게 설정하는 일은 딥러닝 모델 학습을 위한 기본적인 준비 과정입니다.

[표 3-1] 모델링을 위한 데이터 자료형과 차원

구분	x	y(회귀)	y(분류)
자료형	torch.float32(실수형)	torch.float32(실수형)	torch.long 또는 torch.int64(정수형)
차원	2차원 텐서	2차원 텐서(열 1개)	1차원 텐서

make_DataSet 함수를 정의했다면 이제 이 함수를 이용해 데이터 로더와 검증 데이터 세트(텐서)를 준비합니다. 다음과 같이 코드를 작성합니다.

CODE

```
print(type(y_train), type(y_val))
y_train = y_train.values
y_val = y_val.values
print(type(y_train), type(y_val))
```

OUTPUT

```
<class 'pandas.core.series.Series'> <class 'pandas.core.series.Series'>
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
```

① y_train과 y_val을 make_DataSet 함수에 입력하기 전에 넘파이 배열로 변환합니다. 현재 이 데이터들은 판다스의 시리즈 형식입니다. 판다스의 데이터프레임이나 시리즈 형식의 데이터에 values 속성을 적용하면 넘파이 배열로 변환할 수 있습니다.

실행 결과를 보면 판다스 시리즈(pandas.core.series.Series)의 데이터가 넘파이 배열(numpy.ndarray)로 변환되었음을 알 수 있습니다.

이제 make_DataSet 함수로 데이터 로더를 구성합니다.

CODE

```
train_loader, x_val_ts, y_val_ts = make_DataSet(x_train, x_val, y_train, y_val, 32) ①
```

```
# 첫 번째 배치만 로딩해서 살펴보기
for x, y in train_loader: ②
    print(f"Shape of x [rows, columns]: {x.shape}")
    print(f"Shape of y: {y.shape} {y.dtype}")
    break
```

OUTPUT

```
Shape of x [rows, columns]: torch.Size([32, 3])
Shape of y: torch.Size([32, 1]) torch.float32
```

① make_DataSet 함수를 사용해 학습용 데이터 로더(train_loader)와 검증용 텐서(x_val_ts, y_val_ts)를 각각 생성합니다.
② 데이터 로더가 잘 만들어졌는지 확인하기 위해 첫 번째 배치의 구조(shape)를 출력합니다.

실행 결과를 살펴보면 x, y 각각 행(rows)이 32(batch_size의 지정값)임을 확인할 수 있습니다.

make_DataSet 함수의 코드를 보면서 한 가지 의문이 생길 수 있습니다. “왜 학습용 데이터는 데이터 로더로 만들고 검증용 데이터는 그냥 텐서로 반환하지?”라고 말입니다. 학습 과정에서는 전체 데이터를 한꺼번에 처리하지 않고 미니배치 단위로 나누어 학습한다고 했습니다. 따라서 데이터 로더로 선언하고 사용합니다. 반면 검증 과정은 모델의 성능을 확인하는 과정입니다. 이때는 검증 데이터 전체를 한꺼번에 넣어 평가하면 되므로 굳이 배치 단위로 나눌 필요가 없습니다. 다만 검증용 데이터의 크기가 클 경우에는 효율적인 메모리 사용을 위해 데이터 로더를 만들어 사용하는 경우가 있습니다.

모델 구조 설계부터 학습까지

모델 구조 설계

이제 모델 구조를 코드로 구현합니다. 층이 하나인 간단한 형태의 모델 구조를 설계하고 손실 함수와 옵티마이저를 지정하는 코드도 다음과 같이 작성합니다.

CODE

```
n_feature = x.shape[1]
model1 = nn.Sequential(nn.Linear(n_feature, 1)).to(device)
loss_fn = nn.MSELoss()
optimizer = Adam(model1.parameters(), lr=0.01)
print(model1)
```

OUTPUT

```
Sequential(
  (0): Linear(in_features=3, out_features=1, bias=True)
)
```

층 하나, 노드 하나인 모델 구조를 선언했습니다. 모델을 출력(print)하면 입력(feature)은 3개이고 노드(실행 결과에서는 out_feature로 표현됨)는 1개임을 알 수 있습니다. 앞서 모델 구조를 설명할 때 각 층의 출력값 수는 노드 수와 같다고 했습니다.

모델 학습 함수 생성: train

다음은 모델을 학습시키는 단계입니다. 설계된 모델과 손실 함수, 옵티마이저를 사

용해 학습할 차례입니다. 모델을 학습시키는 과정 역시 딥러닝 모델링에서는 늘 반복되는 과정입니다. 따라서 사용자 정의 함수를 하나 만들어 두면 모델을 학습시킬 때마다 반복해 사용할 수 있어 효율적입니다.

모델을 학습시킬 함수는 다음과 같은 절차를 따라야 합니다.

- ① 데이터 로더에서 배치 단위로 데이터를 추출합니다.
- ② 모델에 입력해 예측값을 추출합니다.
- ③ 예측값과 목표값을 비교해 오차를 계산합니다.
- ④ 오차를 줄이는 방향으로 가중치를 업데이트합니다(옵티마이저의 역할).
- ⑤ ①~④를 반복하며 학습 데이터 세트 전체를 배치 단위로 학습합니다.

다음은 모델을 학습시키는 사용자 정의 함수인 `train`의 코드입니다.

```
CODE
def train(dataloader, model, loss_fn, optimizer, device): ①
    size = len(dataloader.dataset) ②
    num_batches = len(dataloader) ③
    tr_loss = 0 ④
    model.train() ⑤
    for x, y in dataloader: ⑥
        x, y = x.to(device), y.to(device) ⑦
        pred = model(x) ⑧
        loss = loss_fn(pred, y) ⑨
        tr_loss += loss ⑩
        loss.backward() ⑪
        optimizer.step() ⑫
        optimizer.zero_grad() ⑬
    tr_loss /= num_batches ⑭
    return tr_loss.item() ⑮
```

- ① `train` 함수는 데이터 로더, 모델, 손실 함수, 옵티마이저, 디바이스를 입력으로 받습니다.
- ② `dataloader.dataset`은 데이터 로더에 담긴 텐서 데이터 세트입니다. `len` 함수로 길이를 계산하면 첫 번째 축(차원)의 데이터 개수를 반환합니다. 보통 데이터 세트의 첫 번째 축은 데이터의 건수입니다.
- ③ `len` 함수에 `dataloader`를 넣고 실행하면 추출할 수 있는 배치 수를 세어 반환합니다. 예를 들어 100건의 데이터를 `batch_size=30`으로 지정하면 반환하는 값은 4입니다. 배치 크기가 30이므로 마지막 배치에는 데이터가 10개밖에 없지만 하나의 배치로 인정됩니다. `Data Loader` 함수 옵션에서 `drop_last=False`가 기본 설정인데, 만약 이 옵션을 `True`로 설정하면 지정한 사이즈보다 작은 배치는 제외합니다.
- ④ 학습 오차를 저장할 변수를 선언하고 초기값으로 0을 지정합니다.
- ⑤ 모델을 학습 모드(Training Mode)로 전환해 학습할 준비를 합니다.

- ⑥ 데이터 로더에서 배치 단위로 x 와 y 를 추출합니다.
- ⑦ 추출한 배치 x 와 y 를 모델과 같은 디바이스에 할당합니다.
- ⑧ x 를 모델 함수에 입력하고 예측을 수행합니다. 예측 결과는 변수 `pred`에 저장합니다.
- ⑨ 손실 함수로 예측값과 목표값 사이의 오차를 계산합니다. 이 오차는 배치 하나의 MSE 값입니다.
- ⑩ 이번 배치에서 계산된 오차를 변수 `tr_loss`에 더합니다.
- ⑪ 역전파를 수행해 모델 각각의 파라미터에 대한 오차의 기울기를 계산합니다.
- ⑫ 기울기를 바탕으로 모델의 파라미터(가중치)를 업데이트합니다. 두 줄(⑪, ⑫)의 간단한 코드지만, 이때 기울기 계산과 함께 오차의 역전파(Back Propagation) 과정이 수행되면서 전체 오차를 줄이는 방향으로 파라미터가 각각 조정됩니다.
- ⑬ 이번엔 사용한 기울기를 초기화합니다. 만약 초기화하지 않으면 이전 기울기가 누적되어 잘못된 학습이 진행될 수 있습니다.
- ⑭ ⑥~⑬ 반복으로 전체 배치를 모두 학습한 다음, 배치 단위로 모두 더한 손실값(`tr_loss`)을 배치 수로 나누어 최종 MSE 값을 계산합니다. 이 값은 학습 데이터 전체를 한 번 학습하고 얻은 손실값입니다.
- ⑮ 텐서로 계산된 손실값 `tr_loss`는 0-way 텐서입니다. 이 값을 텐서가 아닌 스칼라 값으로 만들려면 `item` 메서드를 추가하면 됩니다. 이 손실값을 반환(`return`)합니다.

코드를 한 줄씩 모두 설명하느라 좀 길었습니다. 하지만 딥러닝 모델의 학습 과정을 이해하는 매우 중요한 코드이므로 모르는 부분이 있다면 다시 한번 잘 읽어 보기 바랍니다.

모델 검증 평가 함수: `evaluate`

학습이 모두 끝나면 모델에 대한 검증 평가를 수행합니다. 검증 평가는 에포크마다 수행하는데, 중간 성능을 평가하기도 하고 최종 모델의 성능을 평가하기도 합니다. 검증 평가 역시 딥러닝 모델링에서 늘 반복되는 과정입니다. 따라서 검증 평가를 위한 사용자 정의 함수를 생성합니다. 함수의 이름은 `evaluate`로 하고 다음과 같이 작성합니다.

```
def evaluate(x_val_tensor, y_val_tensor, model, loss_fn, device): ①
    model.eval() ②
    with torch.no_grad(): ③
        x, y = x_val_tensor.to(device), y_val_tensor.to(device)
        pred = model(x)
        val_loss = loss_fn(pred, y).item()
    return val_loss, pred ④
```

- ① evaluate 함수는 x와 y의 검증 텐서 데이터, 모델, 손실 함수, 디바이스를 입력으로 받습니다.
- ② 모델을 학습 모드에서 평가 모드로 변경합니다.
- ③ 평가할 때 with 블록에서는 기울기를 계산하지 않도록 설정합니다. 메모리를 절약하고 평가 속도를 향상하기 위해 반드시 필요합니다.
- ④ 결과로 검증 손실값 val_loss와 예측값 pred를 반환합니다.

사용자 정의 함수 train과 evaluate를 각각 생성했으니 이제 이 함수들을 이용해 보스턴 집값 모델을 학습시키고 검증합니다. 다음 코드를 작성합니다.

CODE

```
epochs = 50 ①
tr_loss_list, val_loss_list = [], [] ②
for t in range(epochs): ③
    tr_loss = train(train_loader, model1, loss_fn, optimizer, device) ④
    val_loss, _ = evaluate(x_val_ts, y_val_ts, model1, loss_fn, device) ⑤
    tr_loss_list.append(tr_loss) ⑥
    val_loss_list.append(val_loss)
    print(f"Epoch {t+1}, train loss: {tr_loss:4f}, val loss: {val_loss:4f}") ⑦
```

OUTPUT

```
Epoch 1, train loss : 575.150574, val loss : 496.143677
Epoch 2, train loss : 563.427124, val loss : 486.102875
Epoch 3, train loss : 550.694763, val loss : 476.235138
...
Epoch 48, train loss : 269.287384, val loss : 201.011337
Epoch 49, train loss : 264.016693, val loss : 197.745712
Epoch 50, train loss : 256.517303, val loss : 194.616913
```

- ① 학습 데이터를 50번 반복 학습할 수 있도록 에포크를 지정합니다.
- ② 학습 오차와 검증 오차를 저장할 빈 리스트를 선언합니다.
- ③ 에포크 수만큼 반복 횟수를 지정합니다.
- ④ train 함수로 학습 데이터 전체를 한번 학습한 다음, MSE 값을 tr_loss에 기록합니다.
- ⑤ evaluate 함수로 검증 데이터의 오차를 계산해 val_loss에 저장합니다. 여기서는 예측값 pred는 사용하지 않을 예정이므로 _로 코드를 작성합니다. _는 함수가 반환하는 값을 사용하지 않을 때 작성하는 코드입니다.
- ⑥ tr_loss_list와 val_loss_list에 학습 오차를 추가합니다.
- ⑦ 에포크마다 학습 오차와 검증 오차를 출력합니다.

코드의 결과를 보면 에포크를 진행할 때마다 train loss와 val loss가 조금씩 줄어들고 있음을 알 수 있습니다.

학습 곡선: 학습 성능의 시각화

학습 곡선(Learning Curve)은 딥러닝 모델의 학습 성능을 평가해 적절한 에포크 수를 결정하는 데 필요한 그래프입니다. 학습 곡선으로 학습 데이터와 검증 데이터의 손실값(Loss) 변화를 시각적으로 확인할 수 있습니다. 정답은 아니지만, 모델 학습이 잘 되었는지 경향을 파악하는 데 매우 유용합니다.

학습 곡선 함수

다음은 학습 곡선을 그리기 위해 미리 생성한 `dl_learning_curve` 사용자 정의 함수입니다. 학습 곡선 역시 모델링에서는 반복해 사용하므로 미리 사용자 정의 함수를 정의해 둡니다.

CODE

```
def dl_learning_curve(tr_loss_list, val_loss_list): ①
    epochs = list(range(1, len(tr_loss_list)+1)) ②
    plt.plot(epochs, tr_loss_list, label='train_err', marker = '.') ③
    plt.plot(epochs, val_loss_list, label='val_err', marker = '.') ④
    plt.ylabel('Loss')
    plt.xlabel('Epoch')
    plt.legend()
    plt.grid()
    plt.show()
```

- ① `dl_learning_curve` 함수는 학습 과정에서 생성한 학습 오차와 검증 오차 리스트를 입력으로 받습니다.
 - ② 에포크를 `[1, 2, 3, ..., n]` 형태의 리스트로 생성합니다.
 - ③ x축을 에포크, y축을 학습 오차로 하는 선 그래프를 그립니다. 값 위치에는 `marker` 속성으로 점(.)을 찍도록 지정합니다.
 - ④ ③번 그래프에 겹쳐 검증 오차 그래프를 그립니다.
- 그 외 하단 코드들은 주로 그래프를 꾸밀 때 사용하는 코드입니다.

이 함수를 이용해 보스턴 집값 예제의 학습 곡선을 그립니다.

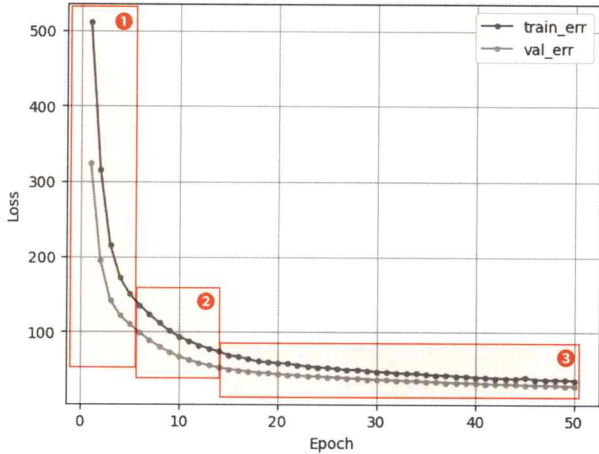
CODE

```
dl_learning_curve(tr_loss_list, val_loss_list)
```

`dl_learning_curve` 함수에는 에포크마다 계산해서 추가한 학습 오차와 검증 오차 리스트를 입력합니다.

[그림 3-6]의 학습 곡선을 보면 에포크가 증가하면서 점차 오차가 줄어들고 있습

니다. 학습 오차는 학습할 때 옵티마이저가 가중치를 조절하기 위해 살펴보는 오차이고, 검증 오차는 에포크마다 학습이 적절히 이루어졌는지 우리가 직접 검증하기 위한 오차입니다. 참고로 [그림 3-6]은 이상적인 학습 곡선의 예시입니다. 독자의 경우 이와 같은 형태의 그래프가 출력되지 않을 수 있습니다.



[그림 3-6] 일반적인(바람직한) 학습 곡선 예시

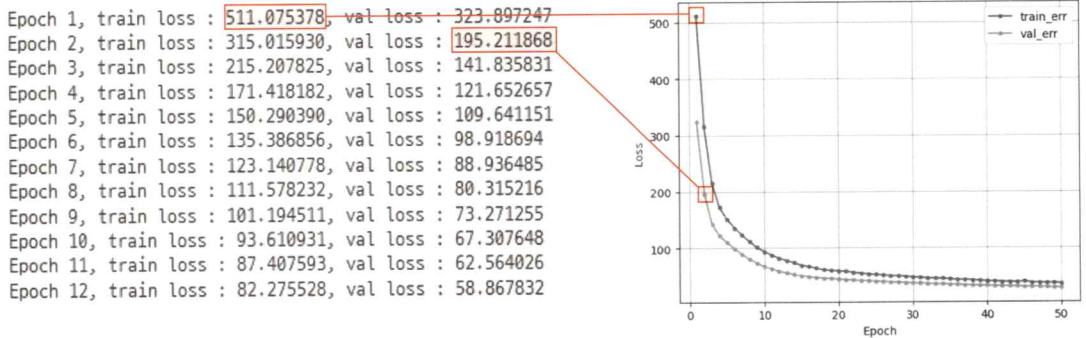
학습 곡선 그래프를 보고 적절히 학습되었는지 파악할 수 있는 세 가지 포인트가 있습니다.

- ❶ 초기 에포크에서 손실값 감소: 학습 초반에는 모델이 데이터의 패턴을 학습하기 시작하면서 학습 오차가 빠르게 감소합니다. 검증 오차도 함께 감소하는 것이 이상적입니다.
- ❷ 손실값 감소 둔화: 급격히 감소하던 손실값은 에포크가 증가함에 따라 둔화됩니다.
- ❸ 손실값 수렴: 에포크가 진행됨에 따라 손실값의 감소율이 점차 줄어들고 학습이 안정화됩니다. 학습 오차와 검증 오차가 수렴하는 시점은 상황마다 다를 수 있습니다.

학습 곡선의 모양은 굉장히 다양한 형태를 띵니다. 따라서 학습 곡선을 보고 바로 이해하는 게 쉽지 않습니다. 학습시킬 때마다 학습 곡선을 확인하고 해석하는 경험 이 많이 필요합니다.

학습 코드의 실행 결과와 학습 곡선 비교

학습 코드를 실행하면 print 문이 에포크마다 학습 오차와 검증 오차를 출력하는데, 이 값들을 그대로 그래프로 옮긴 것이 바로 학습 곡선입니다. [그림 3-7]을 보면 에포크 1의 학습 오차는 511.075378로서 학습 곡선 그래프에서는 학습 오차 선의 첫 번째 값으로 표시됩니다. 에포크 2의 검증 오차는 195.211868로서 그래프에서는 검증 오차 선의 두 번째 값으로 표시됩니다.



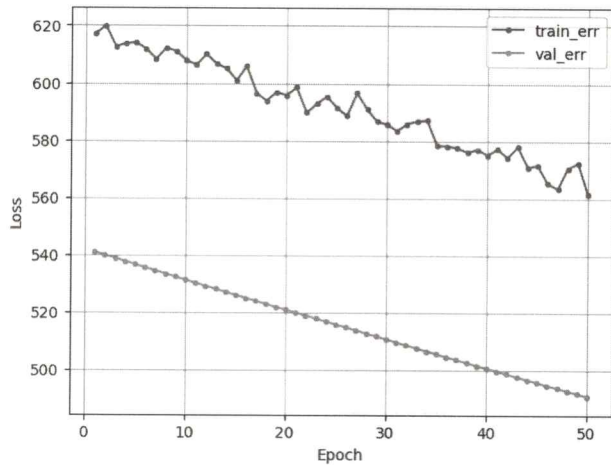
[그림 3-7] 학습, 검증 손실 출력과 학습 곡선

학습 과정에서 에포크마다 손실 함수로 계산된 결과를 그래프로 그대로 옮긴 그래프가 학습 곡선입니다. [그림 3-6] 또는 [그림 3-7]과 같이 초기 에포크에서 오차가 확 줄고 오차 감소세가 꺾이다가 완만해지는 학습 곡선을 보면 학습이 적절하게 진행되었다고 판단할 수 있습니다. 다만 조심할 부분이 있습니다. 학습이 잘 진행되고 있다는 것은 주어진 조건에서, 즉 모델 구조와 지정된 하이퍼파라미터에서 학습이 잘 진행되었다는 의미이지 최적의 모델을 만들었다는 의미는 아닙니다. 최적의 모델은 모델 구조를 잘 설계하고 적절한 하이퍼파라미터를 설정할 때 비로소 가능한 일입니다. 그러므로 학습 곡선은 주어진 상황에서 학습이 잘 진행되었는지 판단하는 도구이지 모델의 최적화를 보장하지는 않는다는 점을 꼭 기억하기 바랍니다.

바람직하지 않은 학습 곡선 예

가장 바람직한 학습 곡선은 앞서 살펴본 [그림 3-7]의 형태입니다. 이번에는 바람직하지 않은 학습 상태의 곡선을 몇 개 살펴보겠습니다.

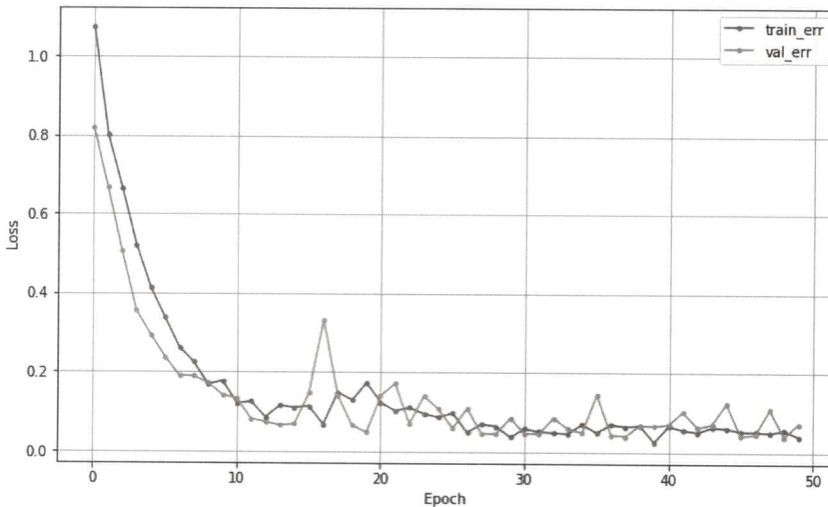
[그림 3-8]를 보면 학습 오차는 에포크가 증가함에 따라 감소하지만, 여전히 큰 값을 유지하고 있습니다. 데이터에 대한 학습이 충분히 이루어지지 않았음을 나타냅니다.



[그림 3-8] 학습이 덜 됨

니다. 검증 오차 또한 지속적으로 감소하고 있으나, 학습 오차와의 간격이 크지 않습니다. 모델이 데이터의 복잡한 패턴을 충분히 학습하지 못했음을 보여 줍니다. 이 학습 곡선은 에포크 수가 부족하거나 학습률이 너무 낮을 때 발생할 수 있습니다.

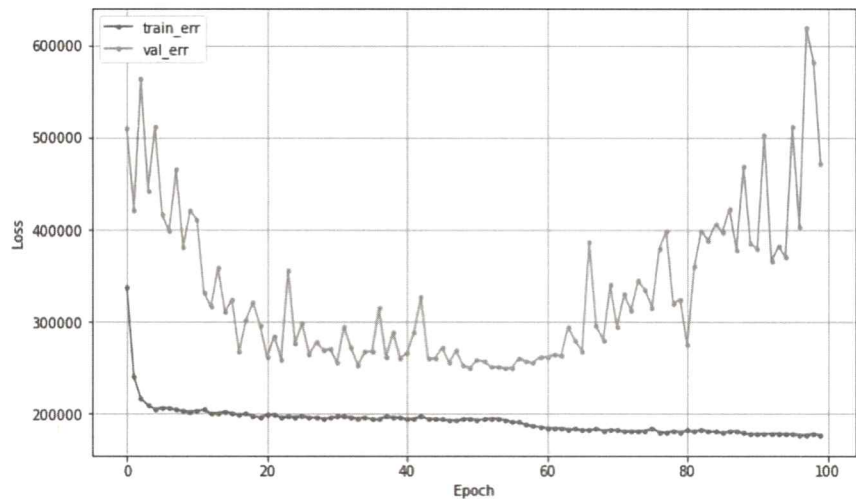
[그림 3-9]는 학습 오차가 들쭉날쭉하게 변동하며 안정되지 않은 상태를 보여 줍니다. 이 현상은 모델의 가중치 업데이트가 불안정하게 이루어질 때 발생하며 학습이 제대로 이루어지지 않았음을 나타냅니다. 이러한 학습 곡선이 나타나는 주된 원인은 다음과 같습니다.



[그림 3-9] 학습 오차가 들쭉날쭉

- 학습률이 너무 큼: 학습률이 지나치게 크면 모델의 가중치가 적절한 방향으로 조정되지 못하고 과도하게 업데이트됩니다. 이로 인해 손실값이 안정적으로 감소하지 못하고 오히려 불규칙하게 변동(줄었다 늘었다)됩니다.
- 미니배치(Mini Batch) 간 데이터 편차: 미니배치 사이즈가 너무 작거나 데이터가 잘 섞이지 않아 미니배치 간 데이터의 편차가 클 경우 학습의 불안정성을 유발할 수 있습니다.

[그림 3-10]은 학습이 반복될수록(에포크가 증가할수록) 학습 오차는 계속 줄어들지만, 검증 오차는 어느 순간부터 점차 증가하는 그래프입니다. 이 현상을 과적합이라고 합니다.



[그림 3-10] 검증 오차가 증가

과적합은 모델이 학습 데이터에 지나치게(Over) 적합되어(fitting) 학습할 때 사용하지 않은 데이터, 즉 검증 데이터에서 성능이 떨어지는 현상입니다. 학습 오차는 계속 줄어들지만, 검증 오차는 일정 시점 이후 오히려 증가하기 시작합니다. 이는 모델이 학습 데이터에 지나치게 의존하고 있음을 나타냅니다.

과적합이 발생하는 주요 원인은 다음과 같습니다.

- 에포크 수 과도: 모델이 너무 많은 에포크 동안 학습해 학습 데이터의 노이즈나 불필요한 패턴까지 학습하는 경우입니다.
- 모델의 과도한 복잡성: 모델의 파라미터 수가 과도하게 많아 학습 데이터에 대

한 적합도는 높아지는 반면, 새로운 데이터에 대한 일반화 성능이 떨어집니다.

- 학습 데이터 부족: 학습 데이터(표본)가 적어 데이터가 전체 데이터(모집단)를 대표하지 못할 때 과적합이 발생할 가능성이 높습니다.

모델의 복잡도와 과적합은 딥러닝 모델 학습에서 중요한 개념입니다. 모델 성능 최적화를 다루는 6일 차에서 다시 자세히 설명합니다.

모델 평가

이제 모델링의 마지막 단계로 검증 데이터 세트로 예측하고 성능을 평가합니다. 학습할 때 에포크마다 검증 평가를 했던 `evaluate` 함수를 이용해 다시 한번 예측한 다음, 예측값을 다양한 성능 지표로 평가합니다. 모델의 성능을 평가하는 코드를 다음과 같이 작성합니다.

CODE

```
_, pred = evaluate(x_val_ts, y_val_ts, model1, loss_fn, device) ①
mae = mean_absolute_error(y_val_ts.numpy(), pred.numpy()) ②
mape = mean_absolute_percentage_error(y_val_ts.numpy(), pred.numpy()) ③
print(f'MAE : {mae}')
print(f'MAPE : {mape}')
```

OUTPUT

```
MAE: 4.252387046813965
MAPE: 0.19883064925670624
```

- ① `evaluate` 함수로 검증 데이터에서 예측값을 추출합니다. `evaluate` 함수는 결과로 두 개의 값을 반환하는데, 두 값은 손실 함수의 결과값과 검증 데이터의 예측값입니다. 여기서는 손실 함수의 결과값이 필요하지 않으므로 `_`로 처리합니다.
- ② `mean_absolute_error` 함수는 `sklearn`에서 제공하는 평균 절댓값 오차(MAE)를 계산하는 함수입니다. `y_val_ts`와 `pred`는 모두 텐서입니다. 따라서 `mean_absolute_error` 함수를 사용하려면 넘파이 배열로 변환해야 합니다.
- ③ `mean_absolute_percentage_error` 함수는 `sklearn`에서 제공하는 평가 함수로 절대 평균 오차율을 의미합니다. 이 함수 역시 넘파이 배열을 입력으로 받으므로 텐서를 넘파이 배열로 변환합니다.

코드 실행 결과를 보면 평균 절댓값 오차(MAE)는 4.252입니다. 집값(1,000달러 단위)을 예측하면 평균적으로 4,252달러 정도의 오차가 발생한다는 의미입니다. 평균 절댓값 오차율(MAPE)은 0.199로서 약 19.9%의 오차가 발생한다는 의미입니다.

지금까지 회귀 모델링의 모든 절차를 코드와 함께 살펴보았습니다. 조금 어려운 부분이 있지만, 그래도 코드의 흐름은 이해가 되지요?

이제 다음으로 중요한 주제인 은닉층을 알아보겠습니다.

회귀 모델링 실습 2: 은닉층이 있는 회귀 모델

앞 절에서는 출력이 하나뿐인 간단한 구조의 딥러닝 모델을 만들었습니다. 이번 절에서는 딥러닝 모델을 보다 복잡한 구조로 설계하고 학습시키겠습니다.

딥러닝의 이름을 보면 뭔가 깊이(Deep) 학습(Learning)한다는 의미를 담고 있는 것 같은데, 정확히 그 의미가 무엇일까요? 딥러닝이라는 이름에는 은닉층(Hidden Layer)을 깊이 쌓는 구조로 학습한다는 뜻이 담겨 있습니다. 딥러닝 모델에서 은닉층은 입력 데이터와 출력층 사이에서 중간 계산을 수행하며 복잡한 패턴을 학습하는 데 중요한 역할을 합니다. 모델 구조에서 은닉층의 노드 수와 활성화 함수를 적절히 조정해 데이터에 최적화된 학습을 수행할 수 있습니다. 따라서 은닉층이 없다면 딥러닝이라는 이름을 붙이기 곤란합니다.

은닉층이 있는 다층 모델 구현하기

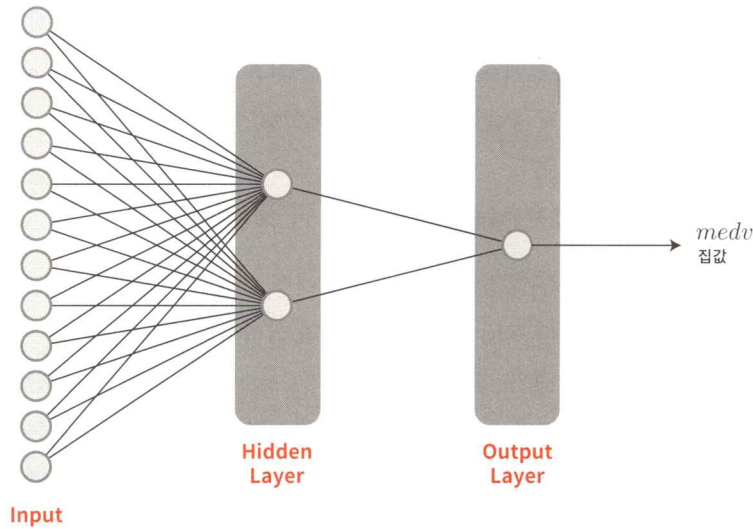
보스턴 집값 데이터를 바탕으로 설계한 [그림 3-11]의 모델은 입력(Input), 은닉층(Hidden Layer), 출력층(Output Layer)으로 구성되어 있습니다. 입력은 12개의 특징(feature)으로 구성되어 첫 번째 층(은닉층)에 연결됩니다.

[그림 3-11]에서 은닉층은 2개의 노드로 구성되어 있습니다. 즉, 입력 데이터를 받아 중간 연산을 수행한 다음, 활성화 함수(Activation Function)로 비선형 변환을 수행하고 다음 층으로 결과를 전달합니다. 은닉층에서 사용하는 대표적인 활성화 함수는 ReLU(Rectified Linear Unit)입니다.

출력층은 1개의 노드로 구성되며 은닉층에서 넘어온 결과를 받아 최종 계산을 한 다음, 예측 결과(여기서는 집값)를 출력합니다.

[그림 3-11]의 구조 그대로 코드로 구현해 은닉층이 있는 회귀 모델을 구현합니다. 앞서 작성했던 '3일차_회귀모델실습1.py'의 코드 파일을 다시 불러와 모델 구조만 다시 코딩하면 바로 구현할 수 있습니다. 하지만 아직 코드에 익숙해져야 하는 입문자를 위해 이번 절에서도 차근차근 처음부터 다시 작성해 보겠습니다.

코드는 다음과 같은 순서로 작성합니다.



[그림 3-11] 은닉층을 포함한 모델 구조

- 환경 준비: 라이브러리 로딩, 디바이스 준비, 사용자 정의 함수 생성
- 데이터 전처리: 데이터 로딩, x와 y 분할, 가변수화, train과 val 분할, 스케일링, 데이터 로더 준비
- 모델 설계, 학습, 평가: 회귀 모델 설계, 손실 함수와 옵티마이저 준비, 학습, 학습 곡선 출력, 예측하고 모델 평가

이런 흐름으로 빠르게 작성합니다. 새 코랩 파일을 생성하고 코드 파일의 이름은 '3일차_회귀모델실습2.py'로 합니다

환경 준비

먼저 환경 준비 코드를 작성합니다. 라이브러리 및 함수 로딩, 디바이스 설정, 그리고 사용자 정의 함수들을 생성합니다.

CODE

```
# 라이브러리, 함수 로딩
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import *
from sklearn.preprocessing import MinMaxScaler
import torch
```

```

from torch import nn
from torch.utils.data import DataLoader, TensorDataset
from torch.optim import Adam

# 디바이스 지정
device = "cuda" if torch.cuda.is_available() else "cpu"

# 함수 생성
## 데이터 로더 선언 함수
def make_DataSet(x_train, x_val, y_train, y_val, batch_size = 32) :
    # 데이터 텐서로 변환
    x_train_tensor = torch.tensor(x_train, dtype=torch.float32)
    y_train_tensor = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)
    x_val_tensor = torch.tensor(x_val, dtype=torch.float32)
    y_val_tensor = torch.tensor(y_val, dtype=torch.float32).view(-1, 1)
    # TensorDataset 생성: 텐서 데이터 세트로 합치기
    train_dataset = TensorDataset(x_train_tensor, y_train_tensor)
    # DataLoader 생성
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle = True)
    return train_loader, x_val_tensor, y_val_tensor

## 학습 함수
def train(data_loader, model, loss_fn, optimizer, device):
    size = len(data_loader.dataset) # 전체 데이터 세트의 크기
    num_batches = len(data_loader) # 배치 크기
    tr_loss = 0
    model.train() # 학습 모드로 설정
    for x, y in data_loader: # batch 단위 로딩
        x, y = x.to(device), y.to(device) # 디바이스 지정
        # Feed Forward(오차 순전파)
        pred = model(x)
        loss = loss_fn(pred, y)
        tr_loss += loss
        # Backpropagation(오차 역전파)
        loss.backward() # 역전파를 통해 각 파라미터에 대한 오차 기울기 계산
        optimizer.step() # 옵티마이저가 모델의 파라미터를 업데이트
        optimizer.zero_grad() # 옵티마이저의 기울기값 초기화.
    tr_loss /= num_batches # 모든 배치에서의 loss 평균
    return tr_loss.item()

## 검증 평가 함수
def evaluate(x_val_tensor, y_val_tensor, model, loss_fn, device):
    model.eval() # 모델을 평가 모드로 설정
    with torch.no_grad(): # 평가 과정에서 기울기를 계산하지 않도록 설정
        x, y = x_val_tensor.to(device), y_val_tensor.to(device)
        pred = model(x)
        eval_loss = loss_fn(pred, y).item() # 예측값 pred와 목표값 y 사이의 오차 계산
    return eval_loss, pred

```

```
## 학습 곡선 함수
def dl_learning_curve(tr_loss_list, val_loss_list):
    epochs = list(range(1, len(tr_loss_list)+1))
    plt.plot(epochs, tr_loss_list, label='train_err', marker = '.')
    plt.plot(epochs, val_loss_list, label='val_err', marker = '.')
    plt.ylabel('Loss')
    plt.xlabel('Epoch')
    plt.legend()
    plt.grid()
    plt.show()
```

함수를 생성하는 코드가 꽤 길지요? 하지만 이처럼 필요한 함수를 미리 생성하면 이후 구현 과정에서는 좀 더 간결한 코드를 작성할 수 있습니다.

데이터 전처리

이어서 데이터 전처리 코드를 작성합니다.

```
CODE
# 데이터 로딩
path = 'https://bit.ly/ds_boston_csv'
data = pd.read_csv(path)

# x, y 분할
target = 'medv'
x = data.drop(target, axis = 1)  # 이번에는 12개의 입력변수 모두를 이용합니다.
y = data.loc[:, target]

# train, val 분할
x_train, x_val, y_train, y_val = train_test_split(x, y, test_size=.2, random_state = 20)

# 스케일링
scaler = MinMaxScaler()
x_train = scaler.fit_transform(x_train)
x_val = scaler.transform(x_val)

# y_train, y_val을 넘파이 배열로 변환
y_train = y_train.values
y_val = y_val.values

# 데이터 로더 준비
train_loader, x_val_ts, y_val_ts = make_DataSet(x_train, x_val, y_train, y_val, 32)
```

모델링을 위한 준비 역시 모두 마쳤습니다. 이어서 은닉층을 포함하는 딥러닝 모델링을 수행합니다.

모델 구조를 코드로 옮기기

은닉층을 포함하고 있는 [그림 3-11]의 모델 구조를 코드로 작성하면 다음과 같습니다.

CODE

```
n_feature = x.shape[1]
model2 = nn.Sequential(nn.Linear(n_feature, 2), ①
                        nn.ReLU(),              ③
                        nn.Linear(2, 1),        ②
                        ).to(device)
```

- ① nn.Linear(n_feature, 2)는 첫 번째 층으로 은닉층입니다. 은닉층은 입력과 출력층 사이에 있는 모든 층을 의미합니다. 12개의 특징(feature)을 입력으로 받으며 노드는 2개가 있습니다.
- ② nn.Linear(2, 1)은 출력층입니다. 2개의 은닉층 노드에서 계산된 값을 입력으로 받아 1개 노드에서 출력값을 만듭니다.
- ③ nn.ReLU: ①의 은닉층 다음에 사용하는 활성화 함수입니다. 이 함수를 사용해 은닉층의 결과를 변환합니다. 활성화 함수는 모델이 데이터의 복잡한 패턴(비선형 패턴)을 학습할 수 있도록 돕습니다.

정리하면 모델 구조는 12개의 입력이 2개의 노드로 이루어진 은닉층을 거쳐 1개의 노드로 이루어진 출력층으로 연결됩니다. 모델을 코드로 구현할 때는 각각의 층은 nn.Linear 함수로 만들고 은닉층 다음에는 활성화 함수 nn.ReLU를 배치합니다. 입력은 별도의 함수를 사용하지 않으며 첫 번째 nn.Linear 함수에서 특징 수(n_features)로 지정됩니다.

모델 학습과 검증 평가

모델 설계를 마쳤으니 손실 함수와 옵티마이저를 지정하고 학습, 학습 곡선, 검증 평가 코드를 작성합니다.

CODE

```
# 손실 함수, 옵티마이저 준비
loss_fn = nn.MSELoss()
optimizer = Adam(model2.parameters(), lr=0.01)

# 학습
epochs = 100      # 100번 반복 학습
tr_loss_list, val_loss_list = [], []
for t in range(epochs):
```

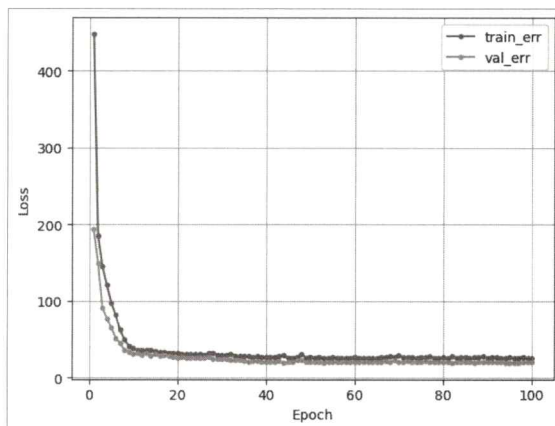
```
tr_loss = train(train_loader, model2, loss_fn, optimizer, device)
val_loss, _ = evaluate(x_val_ts, y_val_ts, model2, loss_fn, device)
tr_loss_list.append(tr_loss) # 학습 곡선을 그리기 위해 리스트에 추가
val_loss_list.append(val_loss) # 학습 곡선을 그리기 위해 리스트에 추가
print(f"Epoch {t+1}, train loss : {tr_loss:4f}, val loss : {val_loss:4f}")
```

학습 곡선

```
dl_learning_curve(tr_loss_list, val_loss_list)
```

OUTPUT

```
Epoch 1, train loss: 447.761414, val loss: 194.380157
Epoch 2, train loss: 185.168839, val loss: 149.917419
Epoch 3, train loss: 145.526703, val loss: 90.958237
...
Epoch 98, train loss: 26.931124, val loss: 20.659004
Epoch 99, train loss: 25.440304, val loss: 20.526436
Epoch 100, train loss: 26.007988, val loss: 20.483324
```



학습 곡선을 보면 초기 에포크에서 손실값이 급격히 감소하다가 8~10번째 에포크에서 오차의 감소 폭이 꺾이고 이후 점차 완만해집니다. 일반적인 학습에서는 이런 형태의 학습 곡선이 나옵니다. 하지만 학습 곡선은 아주 다양하게 나올 수 있음을 꼭 기억하기 바랍니다. 특히 학습 데이터의 건수가 적을 때는 학습시킬 때마다 그래프 모양이 매우 다르게 나옵니다.

이어서 모델의 검증 평가 코드를 작성합니다.

CODE

```
# 예측 결과
_, pred = evaluate(x_val_ts, y_val_ts, model2, loss_fn, device)
# 모델 평가
mae = mean_absolute_error(y_val_ts.numpy(), pred.numpy())
mape = mean_absolute_percentage_error(y_val_ts.numpy(), pred.numpy())
```

```
print(f'MAE : {mae}')
print(f'MAPE : {mape}')
```

OUTPUT

```
MAE : 3.127894878387451
MAPE : 0.14842672646045685
```

은닉층을 포함한 모델의 평가 결과를 보면 평균적으로 약 3,128달러의 오차가 발생하고 오차율은 0.148 정도입니다. 앞서 생성했던 은닉층 없는 모델보다 성능(MAE: 4.252, MAPE: 0.199)이 조금 개선되었다고 보기에는 학습 데이터가 너무 적습니다.

다음 코드를 실행하면 이 예제의 학습 데이터가 총 404건임을 알 수 있습니다.

CODE

```
x_train.shape
```

OUTPUT

```
(404, 12)
```

학습 데이터가 적으면 동일한 모델 구조와 하이퍼파라미터일지라도 학습시킬 때마다 성능 측정값이 들쭉날쭉할 수 있습니다. 마치 여론 조사에서 표본 수가 적으면 조사마다 편차가 많이 차이 나는 것과 마찬가지입니다. 학습 데이터가 수만 건 이상이라면 이런 편차는 많이 줄어들어 동일한 조건에서도 안정적인 성능 결과를 얻을 수 있습니다. 모델의 성능은 6일 차에서 자세히 다룹니다.

추가 학습하기

앞에서 모델을 100번 학습(에포크 100)했을 때 학습 오차(train loss)는 26.007988, 검증 오차(val loss)는 20.483324였습니다. 만약 이 상태에서 다시 학습 코드를 실행하면 모델은 처음부터 다시 학습하는 것이 아니라, 바로 이 시점의 결과(즉, 이미 학습된 가중치)를 이어받아 계속 학습합니다.

완전히 처음부터 학습을 다시 시작하려면 모델을 다시 선언해야 합니다. 모델 선언을 다시 하면 가중치가 무작위로 초기화되기 때문에 이전 학습 결과와 상관없이 처음부터 학습을 다시 시작합니다.

활성화 함수

지금까지 대부분의 코드들은 자세히 설명했지만, 활성화 함수에 대해서는 깊이 있

게 이야기하지 않았습니다. 이번에는 은닉층 다음에 사용하는 활성화 함수를 좀 더 알아보겠습니다.

활성화 함수의 필요성

딥러닝은 여러 은닉층을 쌓아 데이터에 담긴 다양한 패턴을 학습합니다. 이때 각각의 층에서 계산된 결과값을 다음 층으로 전달하기 전에 활성화 함수를 사용해 비선형성을 추가합니다. 비선형성이 추가되면 복잡한 패턴을 학습할 수 있습니다.



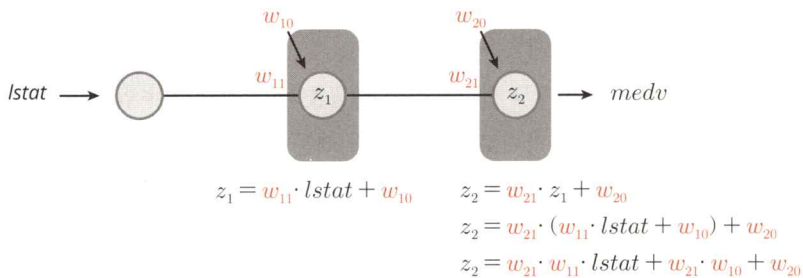
선형성과 비선형성

선형성(linearity) 혹은 선형 관계란 입력이 변할 때 출력이 일정한 비율로 변하는 관계를 말합니다. 예를 들어 $y = 2x$ 는 전형적인 선형 관계입니다. x 가 두 배가 되면 y 도 항상 두 배가 됩니다. 이런 관계는 직선 형태로 표현되며 단순해서 예측하기 쉽습니다.

반면 비선형성(non-linearity) 혹은 비선형 관계는 입력과 출력이 일정한 비율로 변하지 않는 관계를 말합니다. 예를 들어 $y = x^2$ 이나 $y = \sin(x)$ 는 비선형 관계입니다. x 가 두 배가 되더라도 y 가 두 배가 되지 않죠. 이런 관계는 곡선 형태로 표현되며 복잡한 패턴을 나타낼 수 있습니다.

활성화 함수가 없다면 딥러닝 모델은 단순한 선형회귀와 같은 모델로 동작하게 되어 데이터의 복잡한 패턴을 학습할 수 없습니다. 즉, 모델은 선형 함수로만 동작합니다. 이 경우 은닉층을 아무리 추가해도 전체 모델은 선형회귀와 동일한 결과를 보입니다.

예를 들어 [그림 3-12]와 같은 단순한 모델에서 은닉층에 활성화 함수가 없다면 각각의 층은 이전 층의 출력값과 가중치를 단순히 선형적으로 결합합니다. 층이 아무리 많이 쌓여도 결과적으로 식은 $\hat{y} = w \cdot x + b$ 형태의 선형 조합으로서 선형회귀와 동일합니다.



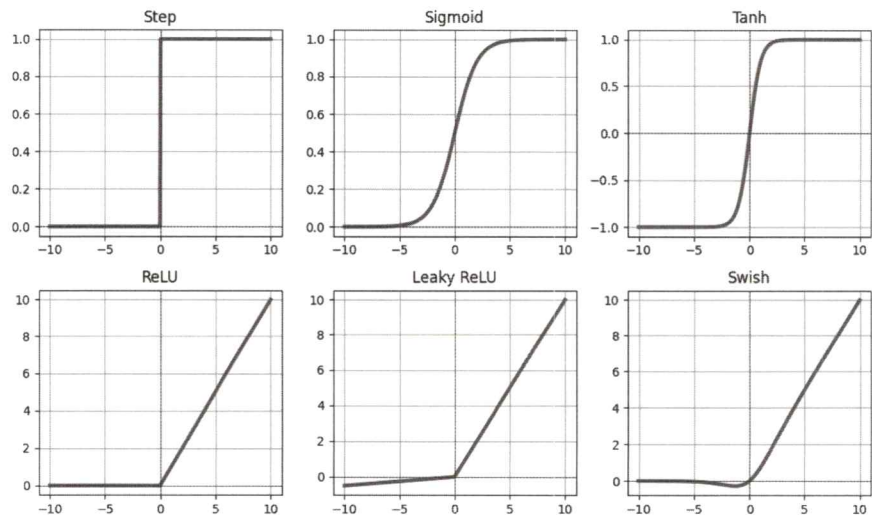
[그림 3-12] 은닉층에 활성화 함수가 없으면 발생하는 문제

이런 모델은 데이터의 복잡한 패턴(비선형 관계)을 학습하지 못합니다. 따라서 딥러닝 모델을 설계할 때는 은닉층에 반드시 활성화 함수를 추가해야 합니다. 활성화 함수를 추가하면 입력 데이터가 각각의 층을 거칠 때마다 단계적으로 연산하면서 더 고차원적인 특징을 학습합니다.

활성화 함수의 종류

성능 최적화를 위한 연구와 함께 활성화 함수도 발전하고 있습니다. 이 중 몇 가지 활성화 함수를 간단히 알아보겠습니다.

- Step 함수: 입력값이 0보다 크거나 같으면 1, 작으면 0을 반환합니다. 초기 뉴럴 넷에서 사용된 활성화 함수로 지금은 잘 쓰이지 않습니다.
- Sigmoid 함수: 입력값이 클수록 1에, 작을수록 0에 근사하도록 변환하는 함수입니다. 주로 이진 분류 모델의 출력층에서 사용됩니다.
- Tanh 함수: 시그모이드 함수와 굉장히 유사한데, 자세히 보면 출력값의 범위가 $(-1, 1)$ 사이입니다. 주로 RNN 계열 알고리즘(RNN, LSTM, GRU 등)의 활성화 함수로 사용됩니다.
- ReLU 함수: 입력이 양수일 때는 그대로 반환하고 음수일 때는 0을 반환합니다. 현재 은닉층에서 가장 많이 사용되는 활성화 함수입니다($\max(0, x)$).



[그림 3-13] 다양한 활성화 함수

- Leaky ReLU 함수: ReLU에서 변형된 활성화 함수로서 입력이 음수일 때도 작은 기울기(Alpha)를 허용합니다. ReLU를 개선한 활성화 함수지만, 성능에서 큰 차이가 없다고 평가되어 ReLU를 대체하지는 못하고 있습니다.
- Swish 함수: 역시 ReLU에서 변형된 활성화 함수로서 입력이 0인 근방에서 바로 꺾이지 않고 부드럽게 처리된 형태로 출력됩니다($x \times \text{sigmoid}(x)$). 가장 최근에 나온 활성화 함수인데, 경우에 따라서는 ReLU보다 나은 성능을 보이기도 합니다.

활성화 함수의 역할

활성화 함수는 은닉층과 출력층에서 각기 다른 목적으로 사용됩니다. 은닉층에서는 비선형 함수로 변환해 모델이 복잡한 데이터 패턴을 학습하도록 돕습니다. 비선형성을 추가해 여러 층이 결합되었을 때는 모델이 계층적인 특징을 학습하도록 합니다.

회귀 모델은 출력층에서 활성화 함수를 사용하지 않지만, 분류 모델은 출력층에 활성화 함수를 사용합니다. 이는 모델의 출력값을 문제 유형(이진 분류, 다중 분류)에 적합한 형태로 변환하기 위함입니다. 이진 분류에서는 시그모이드(Sigmoid) 함수를 이용해 출력값을 0~1 사이의 값(확률값)으로 만듭니다. 다중 분류에서는 소프트맥스(Softmax)를 사용해 출력값을 각 클래스의 확률로 변환합니다. 출력층의 활성화 함수는 분류 모델링에서 다시 자세히 다룹니다.

은닉층과 노드 수를 결정하는 가이드라인

은닉층의 노드 수를 어떻게 설정하는지에 따라 모델의 성능과 복잡도가 달라집니다. 복잡한 데이터 패턴을 학습하기 위해서는 복잡한 모델 구조가 필요합니다. 그러나 어느 정도로 복잡하게 설계하는 것이 맞는지는 실험을 해봐야 알 수 있습니다.

은닉층의 노드 수를 설계하는 데 고려할 점을 먼저 소개하고 기본적인 방향성과 최근 딥러닝 모델 설계의 추세도 함께 설명합니다.

은닉층과 노드 수를 정할 때 고려할 점

은닉층과 노드 수를 정할 때는 다음과 같이 몇 가지 고려할 점이 있습니다.

- 데이터의 특성과 문제 복잡도: 은닉층과 노드의 수는 데이터의 복잡도와 문제 유형에 따라 다르게 지정(설계)하게 됩니다. 데이터가 단순한 선형 관계라면 적은 수의 은닉층과 노드로도 충분합니다. 반면 데이터가 비선형적이고 복잡하다면 더 많은 은닉층과 노드가 필요합니다.
- 과적합(Overfitting) 문제: 은닉층과 노드 수가 지나치게 많으면 모델이 데이터를 과도하게 학습해 새로운 데이터에서 성능이 떨어질 수 있습니다.
- 계산 효율성: 은닉층과 노드 수가 많을수록 계산 비용이 증가하고 학습 시간이 오래 걸립니다. 하드웨어 자원(CPU, GPU 및 메모리)의 한계를 고려해 적절히 설정해야 합니다.

기본적인 설계 방향

가장 적절한 은닉층과 노드의 수를 처음부터 알 수 없으므로 실험과 검증으로 결정해야 합니다. 다음은 이때 사용하는 몇 가지 가이드라인입니다.

- 은닉층: 하나씩 늘려가며 성능이 더 이상 개선되지 않을 때까지 실험합니다. 이때 검증 데이터를 이용한 검증 성능으로 비교해야 합니다.
- 노드 수: 노드 수 역시 은닉층과 유사하게 조금씩 늘려가면서 검증 성능을 평가합니다.
- 은닉층이 여러 개일 때 각각의 노드 수: 입력 특징(feature) 수에서 시작해 출력층 방향으로 갈수록(은닉층1→은닉층2→...→출력층) 노드 수를 줄이는 방식을 주로 사용합니다. 노드 수를 점차 줄이는 이유는 데이터의 주요 특징을 추출하고 점진적으로 요약(추상화)해 마지막 출력층으로 전달하기 위함입니다.

예: 입력층(12개 특징) → 은닉층1(8개 노드) → 은닉층2(4개 노드) → 출력층(1개 노드)

하지만 꼭 이런 방식으로 모델 구조를 설계해야 하는 것은 아닙니다. 특정 문제에서는 은닉층의 노드 수를 다양하게 변형하는데, 늘렸다가 줄이는 구조를 채택할 수도 있습니다. 데이터의 특성과 문제 복잡도에 따라 적절한 구조를 설계할 필요가 있으며 여기에는 답이 정해져 있지 않습니다. 따라서 노드 수를 늘렸다가 점차 줄이는 시도도 해보기 바랍니다.

예: 입력층(12개 특징) → 은닉층1(16개 노드) → 은닉층2(8개 노드) → 출력층(1개 노드)

최근 딥러닝 구조 설계 방향

최근 AI 개발자들이 선호하는 딥러닝 모델 설계 방식은 두 단계로 진행합니다. 먼저 가능한 한 은닉층 수와 노드 수를 늘려 복잡하게 구성한 다음, 규제 기법으로 과적합을 방지합니다.

대표적인 규제 기법으로는 조기 종료(Early Stopping), 드롭아웃(Dropout) 그리고 가중치 규제(Regularization)가 있습니다. 모델이 복잡해지면 자연스럽게 따라오는 문제가 바로 과적합입니다. 복잡한 모델에 다양한 규제 기법을 적용해 과적합을 방지하기 위한 장치를 마련합니다. 과적합과 관련해서는 6일 차 성능 최적화에서 자세히 다룹니다.

정리

이번 차시에서는 딥러닝으로 회귀 모델링을 수행하면서 모델의 구조와 요소를 하나씩 살펴보았습니다.

모델은 회귀 모델(Regression)과 분류 모델(Classification)이 있습니다. 회귀 모델은 숫자를 예측하는 모델로 집값, 주식 가격 등 연속적인 값을 예측합니다. 분류 모델은 범주(Class)를 나누는 모델로서 이메일 스팸 분류, 이미지에서 고양이/개 구분 등이 여기에 해당됩니다.

딥러닝 모델은 기본적으로 입력, 은닉층, 출력층으로 구성됩니다. 각 구성 요소의 역할과 특징은 다음과 같습니다.

- 입력: 데이터를 모델로 전달하는 시작 지점입니다. 입력 특징(feature) 수에 따라 입력 노드의 개수가 결정됩니다. 또한 입력은 계산을 수행하지 않고 데이터를 은닉층으로 전달하는 역할만 합니다.
- 은닉층: 데이터의 특징을 학습하고 입력 데이터를 변환해 출력층에 전달합니다. 비선형성을 도입해 복잡한 패턴을 학습하는 중요한 요소입니다. 은닉층은 여러 층으로 구성될 수 있으며 각 층의 노드 수와 활성화 함수에 따라 모델의 성능은 크게 달라집니다. 비선형성을 추가하기 위해 활성화 함수를 사용하는데, 일반적으로 ReLU를 많이 사용합니다(경우에 따라서는 Sigmoid, Tanh, Leaky ReLU 등

을 선택할 수도 있습니다. 은닉층이 깊어질수록(층의 개수가 많을수록) 모델의 표현력이 증가하지만, 과적합의 위험도 증가합니다.

- 출력층: 최종 예측 결과를 반환합니다. 예측해야 하는 데이터의 형태와 문제 유형에 따라 출력층의 노드 수와 활성화 함수가 달라집니다.

[표 3-2] 회귀 모델의 주요 구성 요소 정리

구분	은닉층	출력층		최적화	
	활성화 함수	활성화 함수	노드 수	옵티마이저	손실 함수
회귀 모델	ReLU	없음	1	Adam	nn.MSELoss

모델 구조를 설계하면서 학습을 위해 준비할 것이 두 가지 더 있었습니다. 바로 손실 함수와 옵티마이저입니다. 모델의 학습 목적은 오차를 최소화하는 가중치를 찾는 일입니다. 최소화할 오차를 계산하려면 손실 함수를 지정해야 합니다. 회귀 모델에서는 평균 제곱 오차(MSE, Mean Squared Error)를, 분류 모델에서는 교차 엔트로피(Cross Entropy)를 주로 사용합니다.

옵티마이저(Optimizer)는 계산된 오차를 최소화하도록 가중치를 조정하는 역할을 합니다. 학습 데이터를 미니배치 단위로 나누어 학습할 때, 각 미니배치마다 오차를 계산하고 그에 따라 가중치를 업데이트합니다. 또한 전체 데이터를 몇 번 반복 학습할지를 결정하는 에포크(Epoch)도 하이퍼파라미터의 하나입니다.

이로써 회귀 모델링의 전체 과정을 모두 살펴보았습니다. 다음 차시에서는 분류 모델링을 수행하며 새롭게 익혀야 할 개념들을 알아보겠습니다.